

Week 3

Learning objectives

- You are able to work with loop structures.
- You are able to use the range function to generate a sequence of numbers.
- You are able to combine different string functions.
- You are able to work with the random module.
- You are able to write simple functions with parameters in Python.
- You are able to use the debugging tools available in your editor.

Agreements

GitHub

All your solutions from week 3 should be posted on Github. To do so, follow the procedure explained on Leho.

To create your Github repository, use this link on Leho

After each exercise, you can do a 'commit & push' so that your version on GitHub is always updated. Give an appropriate message each time.

Continue to work on unfinished exercises at home: regularly perform a 'commit & push' command so that all your solutions are available in your online GitHub repository.

When mastering a programming language such as Python, frequent practice is essential and a prerequisite. For this reason, you will find two additional sections in each lab document. These are labelled as follows

Doing your own research

This part continues from what you learned this week. With a few small coding exercises, you'll practice finding mistakes in code and fixing them.

Every mistake is like a little puzzle. When you look at why something goes wrong, you train important skills like debugging, logical thinking, and spotting patterns. Often, you really understand a programming concept only after you see what happens when you use it the wrong way.

Mistakes also show you how code works "behind the scenes." They help you come up with different solutions and keep trying until it works. This makes you more flexible—and less afraid to test new ideas.

Homework

In this section you will find exercises like those you have already done in the lab.

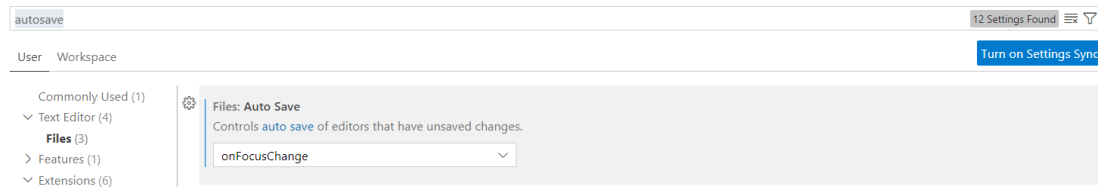
These exercises have the same level of difficulty as the ones you did in the lab. It is only by doing the exercises "on your own" at home that you will consolidate the material.

Stuck on an exercise? Check the theory and see if you can find a similar exercise you did in the lab. Still not succeeding? Bring your preparation to the Study Morning!

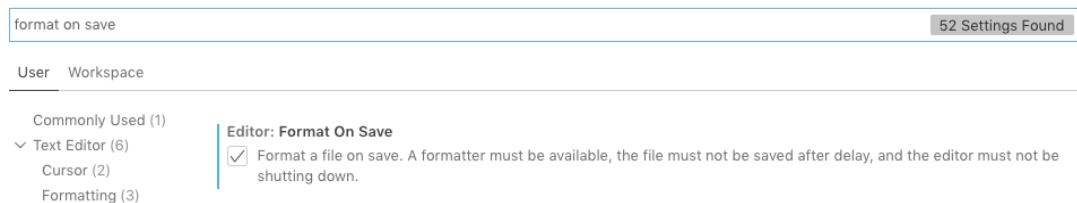
Exercises:

Hints

Make sure that your source file is saved automatically (as soon as the focus changes). Via 'Settings' -> 'Text Editor' -> 'Files: **Auto Save**' you can make sure your file is saved automatically.



You can also automatically **format** your code when saving a file. Via 'Settings' -> 'Editor' -> 'Format on Save' you can make sure your code is formatted correctly right away.



From now on, you only need to press Ctrl+S, and your code will be formatted correctly right away. This reduces the chance of making mistakes and keeps your code organized.

Feedback lab 2

Find and fix the errors in the file **feedback_lab2.py**!

The power of loops

In your comments, always mention the number of the exercise!!

🟡🟢🟣🟤 Exercise 01

Print the sum of the first 100 numbers. Use a for-loop.

```
Sum of 1 until 100: 5050
```

🟡🟢🟣🟤 Exercise 02

Print the list of numbers starting from 20 up to 50. Use a for-loop.

```
20
21
22
23
24
25
26
27
```

🟡🟢🟣🟤 Exercise 03

Print all even numbers from 4 up to and including 100, each on a new line. Use a for-loop.

```
4
6
8
10
12
```

Exercise 04

Print all odd numbers between 10 and 129 side by side. Use a for-loop.

```
11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39, 41, 43, 45, 4
```

Exercise 05

Now count down from 99 down to and including 0, with a step size of -3. Print each number on a new line. Use a for-loop.

```
99
96
93
90
87
84
```

Exercise 06

Ask the user for the following 2 numbers:

- a start value
- a stop value

Print a list of all numbers between the two given limits (limits included) that are divisible by 7 but not a multiple of 5.

```
Enter a start value: 203
Enter a stop value: 508
The numbers you are looking for between 203 and 508 are:
203
217
224
231
238
252
259
```

Exercise 07

Ask the user for the following 3 numbers:

- a starting value
- a positive step size
- how many numbers should be printed

Write a function 'print_list_numbers' that receives these 3 numbers as parameters. The function prints the numbers:

- with the desired amount of numbers,
- where the first number equals the starting value and

- the numbers increase by the step size.

The function has no return value. The printing is done within the function.

```
Enter a start value: 10
Enter a positive step size: 6
How many numbers need to be printed? 5
The numbers you are looking for are:
10
16
22
28
34
```

🔴🟡🟢🟣 Exercise 08

Print the sum of the first 100 numbers. Use a while-loop.
Sum of 1 until 100: 5050

🟢🟡🟣🟤 Exercise 09

Let your application choose a number between 1 and 20 including the limits. Then let the user guess that number. On each attempt, the user gets feedback: "too big" or "too small". The application stops only when the number is guessed. Finally, print the number of attempts.

Tip: use the function 'randint' from the 'random' module:

<https://docs.python.org/3/library/random.html>

```
Guess the number between 1 and 20: 10
The number is too big
Guess the number between 1 and 20: 5
The number is too small
Guess the number between 1 and 20: 7
The number is too big
Guess the number between 1 and 20: 6
Congratulations! You guessed the number in only 4 tries!
```

🟢🟡🟣🟤 Exercise 10

Create an application that asks the user for a number of products. Then ask for each product:

- The category: the user chooses between "Vegetables", "Fruit", "Beverages".
- The price

Finally, print out the total cost for each category, as well as the average price for each category (both with 2 decimals). We use the function 'give_ticket_by_category' for this purpose.

- Parameters are the category name, sum of prices and number of products within these categories
- This function returns a ticket (string) with all the info about the category. Call this function for each category. Keep in mind that a category may contain no products.

An example:

```
Specify the number of products you wish to enter:> 5
What is the category? [V: Vegetables, F: Fruit, B: Beverages]> B
What is the cost price of the product?> 10
What is the category? [V: Vegetables, F: Fruit, B: Beverages]> B
```

```

What is the cost price of the product?> 20
What is the category? [V: Vegetables, F: Fruit, B: Beverages]> V
What is the cost price of the product?> 12.6
What is the category? [V: Vegetables, F: Fruit, B: Beverages]> F
What is the cost price of the product?> 4
What is the category? [V: Vegetables, F: Fruit, B: Beverages]> V
What is the cost price of the product?> 8.7

```

```

2 products are in the category Vegetables
Total cost: 21.30
Average price per product: 10.65

```

```

1 product is in the category Fruit
Total cost: 4.00
Average price per product: 4.00

```

```

2 products are in the category Beverages
Total cost: 30.00
Average price per product: 15.00

```

●●●● Exercise 11

Ask the user for two years. Check every year from the start year up to the end year whether this year is a leap year.

Note: First, create a function 'is_leap_year' that tests whether a year is a leap year or not. Then call this function for each year that lies between the specified limits.

Example:

```

Enter the start year: 1993
Enter the end year: 1997
1993 is not a leap year
1994 is not a leap year
1995 is not a leap year
1996 is a leap year
...

```

Use of String

●●●●● Exercise 12

Ask the user for a word, then print each character one below the other.

```

Enter a word: apple
a
p
p
l
e

```

●●●●● Exercise 13

Write a function 'swap' that receives two strings. The function composes one new string where the first two letters of each word are swapped and both new words are separated by a space.

The result of the function is the new string.

Example: "abc" and "xyz" → "xyc abz"

🟡 Exercise 14

Request the user's last name, first name and date of birth (format: dd-mm-yyyy). Now generate a password as follows:

- the first 3 characters of the last name (in lower case and without spaces)
- the first 2 characters of the first name (in upper case and without spaces)
- 4 digits (mmyy) from the date of birth

Create a function 'generate_password' for this purpose. What parameters do you use, what will be the return value?

Example:

```
Enter your last name: Dewitte
Enter your first name: Marie
Enter your date of birth (dd-mm-yyyy): 30-08-2016
dewMA0816
```

🟡 Exercise 15

Ask the user for their Howest email address. Extract the last name and first name and print them. For simplicity, we assume that the first name consists of one part; the last name can consist of several parts. Make sure every part of the names starts with a capital letter.

```
Enter an existing Howest email address: stijn.walcarius@howest.be
The last name is Walcarius and the first name is Stijn
```

```
Enter an existing Howest email address: charlotte.de.haene@howest.be
The last name is De Haene and the first name is Charlotte
```

🟡 Exercise 16

Create a function 'generate_password_bis' with minimum_length and maximum_length as parameters. The function generates a random password consisting of a combination of lowercase and uppercase letters, and whose length falls between both limits.

Tip: use string.ascii_letters from the module string

Example:

```
Enter the minimum length of the password: 5
Enter the maximum length of the password: 10
Chosen length: 7
Your password is: NNZQaEd
```

🔴🔴🔴🔴 Doing your own research

Find the error(s) in the code examples below. Explain why the code doesn't work as expected. Finally, fix the code.

Tip: afterwards, check your improved code in Visual Studio Code!

Exercise 1

```
n = 100
s = 0
index = 1
while index <= n:
    s = s + index
    index += 1
print(f"Sum of 1 until {n}: {s}")
```

This code doesn't print anything. Can you figure out what the cause is?

Exercise 2

```
program = input("Enter a successful program:> ")
if program == "MCT" and program == "CTAI":
    print("Good choice!")
else:
    print("Better think it over a bit more.")
```

The second print statement is executed every time...? Explain.

Exercise 3

```
def give_new_username(name: str, year: int) -> str:
    return name[0:3] + number

first_name = input("Enter a first name:> ")
number = int(input("Enter a year:> "))
print(f"Your username will be {give_new_username(first_name, number)}")
```

Explain this runtime error and fix the code.

```
return name[0:3] + number
~~~~~^~~~~~
```

TypeError: can only concatenate str (not "int") to str

Exercise 4

```
def count_letter_in_word(word: str, target_letter: str) -> int:
    count = 0
    for index in range(0, len(word)):
        letter = index[0]
        if letter == target_letter:
            count += 1
    return count

print(f"Test for letter e: {count_letter_in_word('orange','e')} times")
```

Explain this runtime error and fix the code.

```
letter = index[0]
~~~~~^^^
```

TypeError: 'int' object is not subscriptable

Homework

●●●● Homework 1

Ask the user for a word. Loop over each character in the word, counting the number of vowels and consonants. Afterwards, print both numbers.

```
Enter a word: apple
Number of vowels: 2
Number of consonants: 3
```

●●●● Homework 2

Ask the user for a word. Loop over each character in the word and replace all vowels with an asterisk (*).

```
Enter a word: apple tree
The string with the vowels replaced is: *ppl* tr**
```

●●●● Homework 3

Write a function `check_email_address(email)` that checks whether a given email address is a valid Howest student email address. A valid address follows the pattern:

`firstname.lastname@student.howest.be`

The function expects one parameter (email of type `str`) and returns a boolean value (True or False).

Examples:

- `check_email_address("jan.janssens@student.howest.be")` → True
- `check_email_address("jan.janssens@howest.be")` → False (staff address)
- `check_email_address("janjanssens@student.howest.be")` → False (no dot between first name and last name)
- `check_email_address("jan..janssens@student.howest.be")` → False (consecutive dots)

Tip: Split the address at the @ symbol, check if the domain matches exactly, and verify that the username part consists of at least two non-empty parts separated by a dot.